Name: Krish Singh
Roll: AID25072
B.Tech Artificial Intelligence & Data Science - 2nd Semester
Object Oriented Programming – Lab Assignment 8

Analysis Questions:

Common Questions

1. What is the main problem observed in Part A across all exercises?

Ans: The main problem observed in Part A across all exercises is that a single generic method is used in the base class, which forces all subclasses to follow the same implementation even when their real world behavior is different resulting in incorrect and unrealistic outcomes.

2. How does an abstract class enforce correct implementation in Part B?

Ans: An abstract class enforces correct implementation in Part B by declaring abstract methods without providing their body part, this compels every subclass to implement these methods according to its own specific requirements. This ensures correctness and realistic outcomes.

3. Why should the base class not have a fixed implementation for all subclasses?

Ans: The base class should not have a fixed implementation for all subclasses because different subclasses represent different real world entities that require unique behaviors and a fixed implementation reduces flexibility and leads to logically incorrect and unrealistic results.

4. What happens if a subclass does not implement an abstract method?

Ans: If a subclass does not implement an abstract method, the program will not compile and will produce an error, unless the subclass itself is declared abstract in which case the responsibility of implementation is passed further down the hierarchy.

5. Which version is easier to extend when adding a new subclass? Justify.

Ans: Part B is easier to extend when adding a new subclass because the abstract class already defines a common structure or a blueprint, and new subclasses only need to implement the required abstract methods without modifying existing code, making the design more scalable and maintainable.

Exercise-Based Questions

Payment System

1. Why is a generic pay() method not suitable for all payment types?

Ans: A generic pay() method is not suitable for all payment types because different payment methods such as UPI and Credit Card require different processing logic, validation steps, and input details, which cannot be handled correctly by a single uniform implementation.

Employee System

2. Why must salary calculation differ between FullTime and Intern?

Ans: Salary calculation must differ between FullTime employees and Interns because full time employees receive additional components such as bonuses and allowances along with their base salary, whereas interns are typically given a fixed stipend without extra benefits.

Notification System

3. Why should each notification type have a different send() implementation?

Ans: Each notification type should have a different send() implementation because communication channels like Email, SMS, and Push notifications use different technologies, protocols, and delivery mechanisms, which require separate handling in each case.

Vehicle System

4. Why is a single fare calculation incorrect in real-world transport systems?

Ans: A single fare calculation is incorrect in real-world transport systems because different types of vehicles have different cost structures, rates, and pricing strategies, which must be handled individually to produce accurate fare calculations.

Exercise 1:
Part A
Sample Code:

```
class Payment{
    double amount;
    void pay(){
        System.out.println("Processing payment of: "+amount);
    }
}
class UPI extends Payment{
    String upiID;
    UPI(double amount, String upiID){
        this.amount=amount;
        this.upiID=upiID;
    }
}
class CreditCard extends Payment{
    String cardNumber;
    CreditCard(double amount, String cardNumber){
        this.amount=amount;
        this.cardNumber=cardNumber;
    }
}
public class PaymentWithoutAbstract {
    public static void main(String[] args) {
        Payment c = new Payment();
        Payment[] arr = new Payment[2];
        arr[0]=new UPI(5000,"1234@oksbi");
        arr[1]=new CreditCard(5000,"BLAIU4AID25072");
        for(int i=0;i<arr.length;i++){
            arr[i].pay();

        }
        c.pay();
    }
}
```

Output:

```
/usr/java/jdk-17-oracle-x64/bin/java -javaagent:/home/krish/.local/share
Processing payment of: 5000.0
Processing payment of: 5000.0
Processing payment of: 0.0

Process finished with exit code 0
```

Part B

Sample Code:

```
abstract class Paymentt{
    double amount;
    Paymentt(double amount){
        this.amount=amount;
    }
    abstract void pay();
}class UPII extends Paymentt{
    String upiID;
    UPII(double amount, String upiID){
        super(amount);
        this.upiID=upiID;
    }
    void pay(){
        System.out.println("UPI Payment: "+amount+" using "+upiID);
    }
}
class CreditCardd extends Paymentt{
    String cardNumber;
    CreditCardd(double amount, String cardNumber){
        super(amount);
        this.cardNumber = cardNumber;
    }
    void pay(){
        System.out.println("Card Payment: "+amount+" using "+cardNumber);
    }
}
public class PaymentWithAbstract{
    public static void main(String[] args){
        // Paymentt c = new Paymentt(6000); using abstract class cant create object of the abstract class
        Paymentt[] arr = new  Paymentt[2];
        arr[0]= new UPII(5000,"1234@oksbi");
        arr[1]=new CreditCardd(5000,"BLAIU4AID25072");
        for(int i=0;i<arr.length;i++){
            arr[i].pay();
        }
    }
}
```

Output:

```
/usr/java/jdk-17-oracle-x64/bin/java -javaagent:/home/krish/.local/share
UPI Payment: 5000.0 using 1234@oksbi
Card Payment: 5000.0 using BLAIU4AID25072

Process finished with exit code 0
```

Exercise 2:
Part A
Sample Code:

```
class Employee{
    double salary;
    void calculateSalary(){
        System.out.println("Salary is: "+salary);
    }
}
class FullTime extends Employee{
    FullTime(double salary){
        this.salary=salary;
    }
}
class Intern extends Employee{
    Intern(double stipend){
        this.salary=stipend;
    }
}
public class SalarySystemNoAbstract {
    public static void main(String[] args) {
        Employee e = new Employee();
        Employee[] arr = new Employee[2];
        arr[0]= new FullTime(4356);
        arr[1]= new Intern(500);
        for(int i=0;i<arr.length;i++){
            arr[i].calculateSalary();
        }
        e.calculateSalary();
    }
}
```

Output:

```
/usr/java/jdk-17-oracle-x64/bin/java -javaagent:/home/krish/.local/share
Salary is: 4356.0
Salary is: 500.0
Salary is: 0.0

Process finished with exit code 0
```

Part B

Sample Code:

```
abstract class Employeee{
    double base;
    Employeee(double base){
        this.base=base;
    }
    abstract void calculateSalary();
}
class FullTimee extends Employeee{
    FullTimee(double base){
        super(base);
    }
    void calculateSalary(){
        System.out.println("FullTime Salary is: "+(base+5000));
    }
}
class Internn extends Employeee{
    Internn(double base){
        super(base);
    }
    void calculateSalary(){
        System.out.println("Internn Stipend is: "+base);
    }
}
public class SalarySystemWithAbstract {
    public static void main(String[] args) {
        // Employeee e = new Employeee(); -> cant create objects of abstract class
        Employeee[] arr = new Employeee[2];
        arr[0]= new FullTimee(4365);
        arr[1]= new Internn(500);
        for(int i=0;i<arr.length;i++){
            arr[i].calculateSalary();
        }
    }
}
```

Output:

```
/usr/java/jdk-17-oracle-x64/bin/java -javaagent:/home/krish/.local/share
FullTime Salary is: 9365.0
Internn Stipend is: 500.0

Process finished with exit code 0
```

Exercise 3:
Part A
Sample Code:

```
class Notification{
    void send(){
        System.out.println("Sending notification");
    }
}
class Email extends Notification{}
class SMS extends Notification{}
class Push extends Notification{}

public class NotificationSystemNoAbstract {
    public static void main(String[] args) {
        Notification e = new Notification();
        Notification[] arr = new Notification[3];
        arr[0] = new Email();
        arr[1] = new SMS();
        arr[2] = new Push();
        for (int i = 0; i < arr.length; i++) {
            arr[i].send();
        }
        e.send();
    }
}
```

Output:

```
/usr/java/jdk-17-oracle-x64/bin/java -javaagent:/home/krish/.local/share
Sending notification
Sending notification
Sending notification
Sending notification

Process finished with exit code 0
```

Part B

Sample Code:

```
abstract class Notificationn{
    abstract void send();
}
class Emaill extends Notificationn{
    void send(){
        System.out.println("Sending Email");
    }
}
class SMSs extends Notificationn{
    void send(){
        System.out.println("Sending SMS");
    }
}
class Pushh extends Notificationn{
    void send(){
        System.out.println("Sending Push Notification");
    }
}

public class NotificationSystemWithAbstract{
    public static void main(String[] args){
        // Notificationn n = new Notificationn(); -> cannot create object of abstract class
        Notificationn[] arr = new Notificationn[3];
        arr[0] = new Emaill();
        arr[1] = new SMSs();
        arr[2] = new Pushh();
        for(int i=0;i<arr.length;i++){
            arr[i].send();
        }
    }
}
```

Output:

```
/usr/java/jdk-17-oracle-x64/bin/java -javaagent:/home/krish/.local/share
Sending Email
Sending SMS
Sending Push Notification

Process finished with exit code 0
```

Exercise 4:
Part A
Sample Code:

```
class Vehicle{
    double distance;
    void fare(){
        System.out.println("Fare: "+distance*10);
    }
}
class Car extends Vehicle{
    Car(double d){
        this.distance=d;
    }
}
class bike extends Vehicle{
    bike(double d){
        this.distance=d;
    }
    void fare(){
        System.out.println("Bike: "+distance*1);
    }
}

public class VehicleFareNoAbstract {
    public static void main(String[] args) {
        Vehicle v = new Vehicle();
        Vehicle[] arr = new Vehicle[2];
        arr[0] = new Car(100);
        arr[1] = new bike(500);
        for(int i=0;i<arr.length;i++){
            arr[i].fare();
        }
        v.fare();
    }
}
```

Output:

```
/usr/java/jdk-17-oracle-x64/bin/java -javaagent:/home/krish/.local/share
Fare: 1000.0
Bike: 500.0
Fare: 0.0

Process finished with exit code 0
```

Part B

Sample Code:

```
abstract class Vehiclee{
    double distance;
    Vehiclee(double distance){
        this.distance = distance;
    }
    abstract void fare();
}

class Carr extends Vehiclee{
    Carr(double d){
        super(d);
    }
    void fare(){
        System.out.println("Car Fare: "+distance*15);
    }
}
class Bike extends Vehiclee{
    Bike(double d){
        super(d);
    }
    void fare(){
        System.out.println("Bike Fare: "+distance*8);
    }
}

public class VehicleFareWithAbstract {
    public static void main(String[] args){
        // Vehiclee v = new Vehiclee(); -> abstract class objects cant be created
        Vehiclee[] arr = new Vehiclee[2];
        arr[0] = new Carr(100);
        arr[1] = new Bike(500);
        for(int i = 0; i < arr.length; i++){
            arr[i].fare();
        }
    }
}
```

Output:

```
/usr/java/jdk-17-oracle-x64/bin/java -javaagent:/home/krish/.local/share
Car Fare: 1500.0
Bike Fare: 4000.0

Process finished with exit code 0
```